

基于 RAG 的 Redis 垂直领域深度问答系统构建

摘要

本文围绕 Redis 技术文档构建了一个垂直领域深度问答系统。系统以 Redis 官方文档和清洗后的课程语料为私有知识库，完成了文档采集、文本清洗、Chunking、Embedding、向量数据库构建、混合检索、上下文增强生成和三维量化评估。针对 Redis 问答中命令名、缩写和概念术语密集的特点，系统采用 Query Rewriting 与 BM25/Vector Hybrid Retrieval 作为高级 RAG 优化策略。生成模块支持 DeepSeek/OpenAI-compatible API，同时保留抽取式生成回退，保证无 API key 环境下也能复现。实验在 10 条 Redis 技术问题上评估 Context Relevance、Faithfulness 和 Answer Relevance，结果分别为 1.0000、0.8116 和 0.8750。结果表明，混合检索能稳定召回相关文档，基于上下文的生成方式能降低幻觉风险，但系统仍受默认 embedding 表达能力和评估集规模限制。

关键词：RAG；Redis；垂直领域问答；混合检索；Query Rewriting；Faithfulness

1. 研究背景

大语言模型具备较强的通用问答能力，但在垂直领域问答中仍存在明显局限。对于 Redis 这类技术文档任务，用户问题往往涉及精确命令、参数语义、系统架构和工程实践。例如，用户可能询问“AOF 和 RDB 的区别”“WATCH 在事务中的作用”“Stream 和 Pub/Sub 的可靠性差异”。这些问题要求答案不仅语言通顺，还必须与官方文档一致。

如果直接让大模型凭参数记忆回答，容易出现三类问题。第一，模型可能混淆不同版本或不同系统的机制。第二，模型可能编造不存在的命令参数或错误使用场景。第三，模型难以给出可追溯来源，用户无法判断答案是否可靠。RAG 技术通过“先检索，再生成”的方式，把大模型回答约束在知识库上下文中，是降低幻觉、提升可解释性的主流方案。

本文选择 Redis 作为垂直领域，是因为 Redis 文档知识边界清晰，同时又包含大量命令名和缩写。这个特点使得 Redis 问答很适合展示 RAG 系统中检索策略的重要性：纯语义相似度可能忽略精确术语，而纯关键词检索又可能无法处理中文提问和英文文档之间的表达差异。因此，本文重点实现并分析 Query Rewriting 与混合检索策略。

2. 任务目标

本项目的目标是构建一个完整、可运行、可解释、可评估的 Redis 垂直问答系统。具体目标包括：

1. 构建 Redis 私有知识库，包含文档收集、清洗和结构化存储。
2. 实现文档 Chunking，把长文档切分为适合检索的片段。
3. 选择并实现 Embedding 模型，将 chunk 存入本地向量数据库。
4. 实现基于检索上下文的答案生成，支持 DeepSeek API 和本地回退。
5. 至少实现一种高级 RAG 优化策略。

- 6. 从 Context Relevance、Faithfulness、Answer Relevance 三个维度进行量化评估。
- 7. 提供完整代码仓库、README、依赖文件和一键推理脚本。

3. 数据集与知识库构建

3.1 数据来源

知识库主题为 Redis 官方文档与技术知识。项目提供两种数据来源。

第一种是内置清洗语料 data/raw/redis_seed_docs.jsonl。它覆盖 Redis 数据类型、过期机制、内存淘汰、持久化、复制、Sentinel、Cluster、事务、Lua 脚本、Pub/Sub 和缓存风险等主题。内置语料保证作业可以离线运行，不依赖网络和外部数据库。

第二种是官方文档采集脚本 scripts/collect_redis_docs.py。该脚本访问 Redis 官方文档页面，去除导航、脚本、样式和重复文本，并将正文清洗为 JSONL 格式。这体现了“自行收集并清理垂直领域文档库”的完整流程。

3.2 文档结构

每条文档保存为 JSONL，字段如下：

```
{
  "doc_id": "redis:persistence",
  "title": "Redis persistence with RDB and AOF",
  "url": "https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/",
  "text": "Redis 通过 RDB 和 AOF 持久化数据..."
}
```

其中 doc_id 用于评估和引用，title 用于展示，url 用于追溯来源，text 是清洗后的正文内容。

3.3 知识覆盖范围

知识库覆盖以下 16 个主题：

类别	主题
基础数据结构	String、Hash、List、Stream、Sorted Set
生命周期管理	Key expiration、TTL、PERSIST
内存管理	maxmemory、LRU、LFU、volatile/allkeys 策略
数据可靠性	RDB、AOF、appendfsync
高可用	Replication、Sentinel
横向扩展	Cluster、hash slot、MOVED/ASK
编程能力	Transaction、WATCH、Lua scripting
消息机制	Pub/Sub、Stream consumer group
缓存工程	缓存穿透、缓存击穿、缓存雪崩

4. 系统总体架构

系统由七个模块组成：文档处理、Chunking、Embedding、向量库、检索、生成和评估。

1. 文档处理模块读取 JSONL 文档并保留来源元数据。
2. Chunking 模块根据 Redis-aware tokenizer 进行重叠分块。
3. Embedding 模块将 chunk 文本映射为定长向量。
4. 本地向量库将 chunk、向量和元数据持久化到 JSONL 文件。
5. 检索模块同时运行 BM25 和向量检索，并融合排序。
6. 生成模块基于 Top-k 上下文生成带引用答案。
7. 评估模块输出三维量化指标和逐样本明细。

整体数据流如下：

```
Redis [ ] -> [ ] JSONL -> Chunking -> Embedding + BM25  
[ ] -> Query Rewriting -> Hybrid Retrieval -> Top-k Contexts  
Top-k Contexts -> DeepSeek/[ ] -> [ ]
```

5. Chunking 与文本处理

技术文档问答的分词策略不能只依赖普通自然语言分词，因为命令名和缩写往往承载关键信息。本项目在 `src/rag_redis/text.py` 中实现 Redis-aware tokenizer，保留三类 token：

1. 英文命令和缩写，例如 AOF、RDB、TTL、EXPIRE、XREADGROUP。
2. 中文领域词，例如“持久化”“主从复制”“消费者组”“缓存击穿”。
3. 中文 n-gram，用于处理中文问题和清洗语料中的短语匹配。

Chunking 默认参数为：

```
chunk_size = 320  
overlap = 40
```

在内置语料中，大多数主题文档长度适中，默认参数可以保留自然段落，避免把答案切成不可读的 token 串。当换成更长的官方文档时，仍可以通过命令行参数调小 chunk size。

6. Embedding 与向量数据库

本项目默认使用 deterministic hashing embedding。它将 Redis-aware token 映射到固定维度向量，并进行归一化。选择这一默认方案主要出于课程复现考虑：

- 不需要下载大模型。
- 不依赖 GPU。

- 不需要外部 embedding API。
- 在不同机器上结果稳定。

其局限也很明确：hashing embedding 缺少深层语义表达能力，不如 BGE、E5 或 SentenceTransformer。真实部署时，可以将 HashingEmbeddingModel 替换为 BAAI/bge-small-zh-v1.5、BAAI/bge-base-zh-v1.5 或其他多语言 embedding 模型。

向量库使用本地 JSONL 文件实现：

```
data/index/vectors.jsonl
```

每条记录保存 chunk 元数据和向量。这样虽然不如 FAISS 或 Chroma 高性能，但结构透明，便于课程作业解释“向量数据库中到底存了什么”。

7. 高级 RAG 优化策略

7.1 Query Rewriting

Redis 官方文档多为英文，而用户问题可能是中文。为了缓解语言差异，系统在检索前进行查询重写。例如：

用户问题关键词	扩展后的检索词
持久化	persistence, RDB, AOF, snapshot, append only file
高可用	replication, sentinel, failover, cluster
过期	expire, TTL, timeout, key expiration
队列	list, stream, consumer group
Lua	eval, script, scripting, atomic

这个策略能提高召回率，尤其适用于“中文提问、英文文档术语”的场景。

7.2 BM25/Vector Hybrid Retrieval

系统同时计算向量相似度和 BM25 分数，并进行归一化融合：

```
score = 0.45 * normalized_vector_score + 0.55 * normalized_bm25_score
```

向量检索负责捕捉语义相关性，BM25 负责捕捉精确术语。Redis 问答中，AOF、RDB、TTL、WATCH 这类词非常关键，因此 BM25 权重略高。

7.3 生成阶段上下文过滤

Top-k 检索结果并不一定全部适合进入生成器。早期实验中，AOF/RDB 问题可能检索到 Pub/Sub 文档，因为 Pub/Sub 文档中出现了“消息不会持久化”。为减少弱相关片段污染答案，生成器只使用与最高检索分数足够接近的上下文。

这个步骤说明：RAG 系统不仅要考虑召回率，还要控制传入 LLM 的上下文质量。

8. 生成模块

生成模块支持两种模式。

第一，DeepSeek/OpenAI-compatible LLM。用户设置 DEEPSEEK_API_KEY 后，系统默认调用 deepseek-v4-pro。系统提示词要求模型只能依据检索上下文回答，如果上下文不足，需要明确说明无法确认，并使用 [1]、[2] 形式引用来源。

第二，抽取式生成器。没有 API key 时，系统从检索上下文中抽取与问题 token 重合度最高的句子，生成可追溯答案。抽取式生成不如 LLM 自然，但能保证作业在离线环境中完整运行。

这种双模式设计兼顾了前沿实践和可复现性。

9. 评估设计

评估集位于 data/eval/eval_questions.jsonl，包含 10 条 Redis 技术问题。每条样本包含 question、gold_doc_ids 和 expected_keywords。

9.1 Context Relevance

Context Relevance 衡量检索结果是否覆盖标注的相关文档：

$$\text{Context Relevance} = |\text{retrieved_doc_ids} \cap \text{gold_doc_ids}| / |\text{gold_doc_ids}|$$

它主要评价检索模块。

9.2 Faithfulness

Faithfulness 衡量答案是否被检索上下文支持：

$$\text{Faithfulness} = \text{supported_answer_tokens} / \text{answer_tokens}$$

它主要评价生成模块是否发生幻觉。

9.3 Answer Relevance

Answer Relevance 衡量答案是否覆盖问题的关键语义点：

$$\text{Answer Relevance} = \text{covered_expected_keywords} / \text{expected_keywords}$$

它主要评价答案是否真正回答了问题。

10. 实验结果

运行命令：

```
python3 evaluate.py --rebuild
```

实验结果如下：

指标	分数
Context Relevance	1.0000
Faithfulness	0.8116
Answer Relevance	0.8750

Context Relevance 达到 1.0000，说明当前知识库规模下，混合检索能够覆盖标注相关文档。Faithfulness 为 0.8116，说明答案多数 token 能从检索上下文中得到支持。Answer Relevance 为 0.8750，说明答案覆盖了大部分期望关键词，但仍有少数问题因为抽取式表达与预期关键词不完全一致而失分。

逐样本观察显示，持久化、过期、Stream/PubSub、Sentinel、事务和 Lua 问题表现较好；Sorted Set 和内存淘汰问题的 Answer Relevance 相对较低，主要因为抽取句未完整覆盖所有期望关键词。

11. 案例分析

以问题“Redis 的 AOF 和 RDB 持久化有什么区别？”为例，系统最终检索到 redis:persistence 文档。答案包含以下要点：

- RDB 会生成某一时刻的数据快照。
- RDB 文件紧凑，适合备份和快速恢复。
- AOF 以追加日志记录写命令。
- AOF 通常能提供更好的数据安全性。
- 生产中可以同时开启 RDB 和 AOF。

这个案例说明，混合检索能够利用 AOF 和 RDB 的精确匹配能力，把问题定位到持久化文档，而不是仅凭“持久化”这个普通词进行模糊匹配。

12. Bad Case 与局限

12.1 弱相关上下文污染

早期版本中，AOF/RDB 问题会把 Pub/Sub 文档排到较前位置，因为 Pub/Sub 文档中也出现了“不会持久化”。这类共现词会造成弱相关上下文污染。解决方式是加入生成阶段的分数阈值过滤，只保留与最高分足够接近的上下文。

12.2 Embedding 能力有限

hashing embedding 可复现，但语义能力较弱。对于复杂同义表达，例如“如何避免热点缓存失效导致数据库压力激增”，系统需要依赖 Query Rewriting 和 BM25，语义泛化仍有限。

12.3 评估集规模有限

内置评估集只有 10 条问题，适合作业展示，但不足以代表真实生产系统。更严谨的评估需要扩展到 50-100 条问题，并加入人工评分或 LLM-as-a-judge。

12.4 生成自然度与忠实度权衡

抽取式生成器更忠实，但表达不够自然；LLM 生成更流畅，但需要更严格的上下文约束和引用检查。

13. 与普通 LLM 问答的比较

普通 LLM 问答依赖模型内部知识，优点是回答自然，缺点是来源不可追溯，且容易出现版本不一致。本文系统的优势在于：

- 答案基于本地 Redis 知识库。
- 每条答案带引用来源。
- 检索结果和分数可观察。
- 可以通过替换知识库适配其他垂直领域。

因此，RAG 更适合技术文档、企业知识库、政策制度、产品手册等需要准确性和可追溯性的场景。

14. 结论

本文完成了一个 Redis 垂直领域 RAG

深度问答系统。系统实现了私有知识库构建、Chunking、Embedding、向量数据库、Query

Rewriting、BM25/Vector Hybrid Retrieval、DeepSeek-compatible

生成和三维量化评估。实验结果表明，混合检索适合 Redis

这类命令密集型技术文档问答，生成阶段的上下文过滤有助于减少弱相关片段导致的答案污染。

后续工作可以从四个方向继续优化：第一，使用 BGE 等神经 embedding 替换默认 hashing

embedding；第二，增加 reranker 对 Top-k 结果二次排序；第三，扩大评估集并引入人工评分；第四，增强 LLM 生成后的引用校验和事实一致性检测。

参考资料

1. Redis Documentation, <https://redis.io/docs/latest/>
2. Redis persistence, https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
3. Redis Streams, <https://redis.io/docs/latest/develop/data-types/streams/>
4. Redis Cluster specification, https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

5. Robertson, S. and Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond.
6. Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.